Madhu-Marga

Digital Beekeeper's Diary – AI-Guided Hive Management App

Project Title

Android App Development using GenAI – Madhu-Marga (Agriculture) – AI-Guided Beekeeping & Hive Management App

Project Number

64

Platform

Android (Kotlin + XML Layouts)

Domain

Agriculture / Beekeeping / Rural Livelihood

Category

Android Application with Room Database, WorkManager & Decision Matrix Engine

Report Type

Internship / Academic Project Report

PREPARED BY:

ARSHID AHMAD MALIK
USN: 1MJ23EC401
1MJ23EC401@MVJCE.EDU.IN

MVJ COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
2025 – 2026

1. Introduction

Madhu-Marga is an Android application developed to help small-scale beekeepers, honey farmers, and agricultural hobbyists across India digitally manage their hive health, log inspection observations, track honey harvests, and receive AI-guided intervention recommendations. The application name blends Sanskrit and Kannada — Madhu (honey) and Marga (path or guide) — reflecting its purpose as the trusted digital companion on every beekeeper's journey.

Developed using Kotlin with XML-based layouts, the app integrates Room Database for permanent local storage of hive records and inspection logs, WorkManager for scheduled background health-check alerts, and a Decision Matrix Engine that analyses logged observations to suggest timely hive interventions. The application targets the agriculture and allied-sector economy — a segment that has historically relied on experience-based guesswork, handwritten diaries, and informal knowledge networks to manage bee colonies.

The application addresses four critical needs of small-scale beekeepers: maintaining a digital register of all hives tagged by ID and location, logging structured inspection checklists (queen presence, pest sightings,

activity level, honey flow), tracking harvest quantities over seasons and years, and receiving data-driven intervention alerts when hive health indicators fall below acceptable thresholds. A PIN-based login system ensures that sensitive hive and harvest data remains secure even on shared or family-use devices.

Madhu-Marga is designed to function entirely offline, requiring no internet connection for its core operations. Data is stored persistently on-device using Room Database, ensuring that hive records and inspection logs survive app restarts, device reboots, and the long gaps between seasonal inspections. The app targets the growing community of hobby beekeepers, farmer-entrepreneurs, and agriculture students across rural Karnataka and India who lack access to specialised hive management software.

2. Problem Statement

Beekeeping is one of India's most promising secondary agricultural activities, supporting rural livelihoods through honey production, beeswax, and pollination services. However, bee colonies are highly sensitive to environmental conditions, pests, and queen health. Beginner and intermediate beekeepers frequently lose entire colonies because they fail to recognise the early signs of a failing hive. Several systemic problems affect this segment:

Challenge	Description
No Structured Inspection Records	Beekeepers rely on memory or unstructured handwritten notes to track hive observations, leading to missed warning signs and delayed interventions.
Colony Loss Due to Late Diagnosis	Without a decision-support tool, beginners often identify hive health problems — mite infestations, queen failure, absconding — too late to save the colony.
No Harvest Benchmarking	Beekeepers have no systematic way to compare honey yields across hives or seasons, making it impossible to identify high-performing hives or optimal harvest windows.
Lack of Forage Guidance	Beginners are unaware of local floral calendars — which flowers bloom when — resulting in suboptimal hive placement and missed nectar flows.
Knowledge Gap	Expert beekeeping knowledge is held by a small number of experienced practitioners. New beekeepers lack access to contextual guidance at the moment of inspection.

These interconnected challenges highlight the urgent need for a dedicated, offline-first digital hive management application that beekeepers can use with minimal technical knowledge, on low-cost Android smartphones, in fields and apiaries far from internet connectivity.

3. Project Vision

Madhu-Marga envisions a world where every beekeeper — whether a seasoned honey farmer managing fifty hives or a student tending their first colony — has access to a reliable, intelligent, and field-ready hive management tool in their own language.

The project is built on four foundational pillars:

- **Accessibility** – Hive management tools must work on affordable Android devices in remote agricultural settings without internet. The app is designed for minimum API 21 devices and operates fully offline for all core features.
- **Simplicity** – The interface must be usable by farmers with limited digital experience.

Checklist-based inspections, large touch targets, and clear iconography minimise the learning curve during a hive visit.

- **Reliability** – Inspection logs and harvest records must never be lost. Room Database provides ACID-compliant local storage that survives app crashes, device reboots, and months of seasonal inactivity between inspections.
- **Intelligence** – The burden of interpreting hive data must be reduced through technology. A Decision Matrix Engine analyses logged observations and automatically generates actionable intervention alerts — harvest now, split the colony, treat for mites — removing the need for expert consultation for routine decisions.

Madhu-Marga is not merely a digital diary; it is an agricultural empowerment tool designed to bring the discipline of data-driven hive management to the informal beekeeping economy, one inspection at a time.

4. Scope of the Application

4.1 Target Users

- Small-scale honey farmers and beekeeping entrepreneurs across rural Karnataka and India
- Agriculture students and hobbyists managing 1–20 hives as a supplementary income source
- Self-help group (SHG) members participating in government beekeeping livelihood schemes
- Any beekeeper who currently records hive observations in a handwritten notebook

4.2 Platform

- Android smartphones with minimum API Level 21 (Android 5.0 Lollipop and above)
- Optimised for low-end and mid-range devices with limited RAM and storage
- APK size kept minimal to support devices with limited internal storage and slow downloads

4.3 Offline Functionality

- All hive, inspection, and harvest data is stored locally using Room Database — no cloud or serverrequired
- Core features including hive registration, inspection logging, harvest tracking, and flora calendarwork fully offline
- Intervention alerts use the device's WorkManager, requiring no internet connectivity

4.4 Key Operational Scenarios

- Registering a new hive with a unique ID, apiary location, colony origin, and installation date
- Logging a structured hive inspection checklist with queen status, pest observations, activity level,and honey flow rating
- Recording a honey harvest with date, hive ID, quantity collected, and quality notes
- Viewing a year-over-year harvest comparison chart for each hive

- Receiving an intervention alert when the Decision Matrix detects low activity or other distressindicators
- Consulting the Flora Calendar for monthly guidance on nearby blooming plants that support beeforage

5. Objectives

The project is aligned with the following measurable objectives:

- **Provide a structured hive registration system** – Ensure that a new hive can be registered within one screen interaction from the home dashboard, with fields for hive ID, location, colony origin, and installation date.
- **Deliver a guided inspection checklist** – Present beekeepers with a standardised set of inspection checkpoints (queen seen, eggs present, capped brood, pest sightings, activity level, honey flow) to ensure consistent and complete data capture.
- **Enable data-driven intervention alerts** – Implement a Decision Matrix Engine that analyses logged inspection data and triggers intervention alerts (treatment required, harvest recommended, split advised) based on predefined threshold rules.
- **Track and compare harvest yields** – Maintain a time-stamped harvest log per hive and display a year-over-year comparison to help beekeepers identify peak harvest windows and high-performing colonies.
- **Provide seasonal forage guidance** – Deliver a Flora Calendar that lists locally relevant blooming plants by month, helping beekeepers optimise hive placement and anticipate nectar flows.
- **Ensure permanent, offline data persistence** – All hive, inspection, and harvest records must survive application restarts, device reboots, and extended periods of inactivity using Room Database.
- **Protect sensitive farm data** – Implement PIN-based login so that only the authorised beekeeper can access hive records and harvest history.

6. Features

6.1 Core Features

Feature	Description
PIN-Based Login	4-digit PIN authentication screen shown at app launch. First-time users create a PIN; returning users enter their saved PIN. Supports change PIN via Settings.
Professional Dashboard	Home screen displays total hive count, number of inspections due, and last harvest summary in real-time cards alongside quick-action buttons.
Hive Register	Register each hive with a unique ID, apiary location tag, colony origin (swarm/package/split), installation date, and optional notes. Stored in Room Database with auto-generated timestamp.
Inspection Log	Structured checklist: Queen seen (Yes/No), Eggs present, Capped brood, Pest sightings (Varroa/SHB/Wax moth), Activity level (High/Medium/Low), Honey flow (Strong/Moderate/Weak/Nil). Each log is timestamped and linked to a hive.
Decision Matrix Engine	Analyses each saved inspection log against a rule set. Triggers coloured intervention alerts: Red (Immediate action — treat/re-queen), Amber (Monitor closely — split/supplement feed), Green (Healthy — harvest window open).
Harvest Tracker	Log honey harvest per hive with date, quantity (kg), quality grade, and notes. Displays a Honey Flow Season progress bar and year-over-year comparison for each hive.
Flora Calendar	Month-by-month guide to locally relevant flowering plants that support bee forage. Helps beekeepers anticipate nectar flows and plan hive relocations.
WorkManager Alerts	Schedules periodic background checks. Sends a push notification if any hive has not been inspected within a user-configured interval (default: 14 days).
Beekeeper Profile	Save beekeeper name, apiary name, location, phone, and years of experience. Apiary name is displayed in the main header.
Settings Screen	Centralised settings for inspection reminder interval, notification toggle, change PIN, and data management preferences.
Logout	Securely log out from the 3-dot menu with a confirmation dialog. Session state is cleared and the login screen is shown on next launch.

6.2 Future Scope Features

- Weather API Integration – Correlate hive activity data with local weather conditions for enhancedintervention recommendations.
- Image Attachment – Allow beekeepers to attach camera photos to inspection logs for visualdocumentation of pest damage or queen cells.

- Multi-Language Support – Extend the interface to support Kannada, Hindi, Tamil, and Telugu inaddition to English.
- Cloud Backup – Optional encrypted cloud sync via Firebase to protect data against device loss.
- Hive-wise Trend Charts – Graphical visualisation of activity level and honey flow trends acrossmultiple inspections.
- Export to PDF – Generate a formatted PDF inspection report for a selected hive and date range forsharing with agriculture extension officers.

7. Functional Requirements

The following functional requirements define the core behaviours the system must support:

FR-1: Authentication

- The system shall display a PIN entry screen on every app launch.
- First-time users shall be prompted to create a 4-digit PIN stored in SharedPreferences.
- Subsequent users shall be required to enter their saved PIN to access the dashboard.
- An incorrect PIN entry shall display an error message and clear the input field.

FR-2: Hive Registration

- The system shall provide input fields for hive ID, apiary location, colony origin, installation date, andnotes.
- The system shall validate that hive ID and location fields are not empty before saving.
- The system shall reject duplicate hive IDs and display an appropriate error message.
- On successful save, the hive record shall be stored in Room Database with a formatted timestamp.

FR-3: Inspection Logging

- The system shall present a structured checklist for each inspection: queen seen, eggs present,capped brood, pest type, activity level, and honey flow rating.
- The system shall validate that at least the activity level field is completed before saving.
- On successful save, the log shall be stored in Room Database linked to the selected hive ID with atimestamp.
- The system shall immediately pass the saved log to the Decision Matrix Engine for analysis.

FR-4: Decision Matrix Engine

- The system shall analyse each inspection log against a predefined rule matrix upon save.
- The rule matrix shall evaluate: activity level (Low triggers Red alert), queen absence combined withno eggs (triggers Red alert), pest sighting (triggers Amber alert), and strong honey flow (triggers Green harvest alert).
- The system shall display the intervention alert on the inspection detail screen with a colour indicatorand recommended action text.
- Alerts shall be stored with the inspection log and visible in the hive history view.

FR-5: Harvest Tracking

- The system shall allow the user to log a harvest entry with date, hive ID, quantity in kilograms, andquality grade.
- The system shall display a Honey Flow Season progress bar showing cumulative yield against theseasonal target.
- The system shall display a year-over-year comparison showing this season's yield versus theprevious season for each hive.

FR-6: Flora Calendar

- The system shall display a month-wise guide to locally relevant flowering plants.
- Each month entry shall list the plant name, bloom period, forage type (nectar/pollen/both), and abeekeeping tip.
- The calendar shall highlight the current month automatically on opening.

FR-7: Inspection Reminder Notifications

- The system shall use WorkManager to schedule periodic background checks at a user-configuredinterval.
- If any hive has not been inspected within the configured interval, a push notification shall be postednaming the overdue hive.
- The user shall be able to configure the reminder interval (7, 14, or 21 days) from the Settings screen.

FR-8: Profile and Settings

- The user shall be able to enter and save beekeeper name, apiary name, location, and phonenumber.
- The apiary name shall be displayed in the main dashboard header.
- The user shall be able to change their PIN and configure the inspection reminder interval from theSettings screen.

8. Non-Functional Requirements

Category	Requirement	Acceptance Criterion
Performance	Load Time	Dashboard must fully render within 800 ms of login on a mid-range Android device.
Performance	Inspection Save	A new inspection log must be saved and the Decision Matrix alert generated within 600 ms of tapping Save.
Usability	UI Simplicity	Any user must be able to log a complete hive inspection within 5 taps from the login screen.
Usability	Legibility	All primary text must use a minimum font size of 13sp. Summary card values must use a minimum of 24sp.
Reliability	Data Persistence	100% of hive, inspection, and harvest records must be retained after app closure, device restart, and 90+ days of inactivity.
Reliability	Crash Rate	Application crash rate must not exceed 1% during standard hive registration, inspection logging, and harvest recording flows.
Accessibility	Touch Targets	All interactive elements must have a minimum touch target size of 48x48dp per Material Design guidelines.
Compatibility	Android Versions	Application must support Android 5.0 (API 21) through Android 14 (API 34).
Security	Data Privacy	All hive and harvest data remains on-device. No data is transmitted to external servers. PIN is stored locally in SharedPreferences.
Offline Support	Core Features	All core features (hive register, inspection log, harvest tracker, flora calendar, alerts) must function without any network connection.

9. System Architecture

Madhu-Marga follows a single-tier, client-only architecture. All data processing, storage, and business logic occur entirely on-device. There are no backend servers, cloud databases, or API dependencies required for the application's core operation.

9.1 Technology Stack

Layer	Technology	Purpose

Language	Kotlin	Primary development language; concise, null-safe, fully interoperable with Android SDK.
UI Framework	XML Layouts + AppCompat	Traditional View-based UI with RecyclerView, CardView, and Material components for broad device compatibility.
Local Database	Room Database (SQLite)	ACID-compliant ORM over SQLite. Provides permanent on-device storage with DAO-based queries and migration support.
Decision Engine	Decision Matrix (Kotlin)	Rule-based evaluation engine that analyses inspection log fields and returns a colour-coded intervention alert with recommended action.
Background Tasks	WorkManager	Schedules and executes periodic hive inspection reminder tasks reliably, surviving device reboots and app closures.
Notifications	NotificationManager	Posts inspection overdue alerts and intervention notifications. Supports Android 8+ notification channels.
Session Storage	SharedPreferences	Stores PIN, login state, beekeeper profile, and reminder interval settings as lightweight key-value pairs.
Build System	Gradle (Groovy DSL)	Dependency management, build configuration, and APK generation including kotlin-kapt for Room annotation processing.
Min. SDK	API Level 21	Targets approximately 98% of active Android devices globally, including low-cost smartphones in rural India.

9.2 Architectural Overview

The application follows a simplified MVC (Model-View-Controller) pattern appropriate for a single-activity, XML-layout Android app:

- **Model Layer** – The data package contains Room @Entity classes (Hive, InspectionLog, HarvestEntry), @Dao interfaces (HiveDao, InspectionDao, HarvestDao), and the @Database singleton (AppDatabase). This layer owns all data access logic and migration handling.
- **View Layer** – XML layout files define the UI structure (activity_main.xml, activity_login.xml, activity_hive_form.xml, activity_inspection.xml, activity_harvest.xml, activity_flora_calendar.xml, activity_settings.xml). Custom drawables provide alert badge and card visual styling.
- **Controller Layer** – Activity classes (MainActivity, LoginActivity, HiveFormActivity, InspectionActivity, HarvestActivity, FloraCalendarActivity, SettingsActivity) orchestrate user interaction, invoke database operations on background threads via Executors, and update the UI via runOnUiThread.
- **Decision Engine** – DecisionMatrix.kt is a stateless Kotlin object that accepts an InspectionLog and returns an AlertResult (colour, title, recommendation string). It is called synchronously within the background save thread after each inspection is stored.

- **Background Workers** – InspectionReminderWorker (Worker) queries the database for hives not inspected within the configured interval and dispatches overdue inspection notifications via WorkManager.

9.3 Database Design

The Room database contains three tables — hives, inspection_logs, and harvest_entries — with the following schemas:

Column	Type	Constraint	Description
id	INTEGER	PRIMARY KEY, AUTO	Auto-generated unique identifier for each hive record.
hiveCode	TEXT	NOT NULL, UNIQUE	User-assigned hive identifier (e.g., H-01, Apiary-North-3).
location	TEXT	NOT NULL	Apiary location or GPS description for field navigation.
colonyOrigin	TEXT	DEFAULT ''	Colony source: Swarm, Package, Split, or Nucleus.
installDate	TEXT	NOT NULL	Date of colony installation in formatted string.
notes	TEXT	DEFAULT ''	Free-text notes about the hive's history or special characteristics.
createdAt	TEXT	NOT NULL	Formatted timestamp of hive record creation.

The inspection_logs table additionally records hiveId (foreign key), queenSeen, eggsPresent, cappedBrood (BOOLEAN fields), pestType (TEXT), activityLevel (TEXT: High/Medium/Low), honeyFlow (TEXT: Strong/Moderate/Weak/Nil), alertColour (TEXT), alertRecommendation (TEXT), and inspectionTimestamp (TEXT NOT NULL). The harvest_entries table records hiveId, harvestDate, quantityKg (REAL), qualityGrade (TEXT), and notes (TEXT).

10. User Flow

The application is designed to minimise friction during field inspections. The complete user journey from app launch to logged inspection with intervention alert is described below:

#	Screen	User Action & System Response
1	**Login Screen**	App launches. If first use, beekeeper creates a 4-digit PIN. On subsequent launches, beekeeper enters their PIN to authenticate.
2	**Dashboard**	Beekeeper sees the main dashboard with summary cards: total hives, inspections due, and last harvest date. Apiary name appears in the header.

3	**Hive Register**	Beekeeper taps + to register a new hive. Fills in hive code, location, colony origin, and install date. Taps Save. Hive appears in the list.
4	**Inspection Log**	Beekeeper selects a hive and taps Log Inspection. Completes the checklist: queen seen, brood, pests, activity level, honey flow. Taps Save.
5	**Intervention Alert**	Decision Matrix analyses the log. A coloured alert card appears instantly: Red (treat now), Amber (monitor closely), or Green (harvest window open).
6	**Hive History**	The hive detail view shows all previous inspection logs in reverse chronological order with their alert colours and timestamps.
7	**Harvest Tracker**	Beekeeper taps Log Harvest, selects hive, enters date, quantity, and quality grade. The Honey Flow Season progress bar updates and year-over-year comparison is shown.
8	**Flora Calendar**	Beekeeper taps 3-dot menu → Flora Calendar. Current month is highlighted. Each entry shows plant name, bloom period, and a seasonal beekeeping tip.
9	**Inspection Reminder**	WorkManager fires at the configured interval. If a hive is overdue for inspection, a push notification names the hive and prompts the beekeeper to visit.
10	**Logout**	Beekeeper taps 3-dot menu → Logout. Confirmation dialog shown. On confirm, session is cleared and login screen is presented.

11. Technical Implementation

11.1 Room Database & Migration

Hive, inspection, and harvest records are stored using the Room persistence library. The HiveDao interface exposes getAll() (ordered by hiveCode ASC), getById(), insert(), update(), delete(), and getHivesDueForInspection(cutoffTimestamp). The InspectionDao exposes insert(), getByHiveId() (ordered by inspectionTimestamp DESC), and getLastInspectionDate(hiveId). The HarvestDao exposes insert(), getByHiveId(), and getYearlyTotals(hiveId) for the year-over-year comparison. The AppDatabase singleton uses double-checked locking to ensure a single database instance throughout the app lifecycle.

A database migration (version 1 to version 2) handles the addition of the alertColour and alertRecommendation columns to the inspection_logs table, ensuring that existing users who upgrade do not lose their inspection history. The migration uses ALTER TABLE statements registered via addMigrations() on the RoomDatabase.Builder.

11.2 Decision Matrix Engine

DecisionMatrix.kt is a stateless Kotlin singleton object exposing a single evaluate(log: InspectionLog): AlertResult function. The Decision Matrix applies a priority-ordered rule set: (1) If activityLevel == LOW and queenSeen == false and eggsPresent == false, return RED alert ('Colony in distress — inspect for queen failure or absconding. Consider emergency re-queening.'). (2) If pestType is not empty (mite or beetle sighting), return RED alert ('Pest detected — treat immediately with approved miticide or SHB trap.'). (3) If

activityLevel == LOW but brood indicators are positive, return AMBER alert ('Activity low — supplement feed and re-inspect in 7 days.'). (4) If honeyFlow == STRONG and cappedBrood is healthy, return GREEN alert ('Honey flow active — prepare supers for harvest within 14 days.'). (5) Otherwise return GREEN ('Hive appears healthy — continue routine monitoring.').

11.3 Background Threading

Room does not permit database operations on the main (UI) thread. All database calls are executed using a single-threaded Executors.newSingleThreadExecutor() instance. The Decision Matrix evaluation is performed within the same background thread immediately after the inspection log is saved. The AlertResult is posted back to the UI thread via runOnUiThread{}, which updates the alert card's background colour and text. This pattern avoids the need for LiveData or coroutines while maintaining thread safety.

11.4 WorkManager for Inspection Reminders

InspectionReminderWorker extends Worker and implements doWork() to perform the periodic overdue inspection check. It is enqueued as a PeriodicWorkRequest with an interval matching the user's configured reminder setting (7, 14, or 21 days) using WorkManager.enqueueUniquePeriodicWork() with ExistingPeriodicWorkPolicy.REPLACE, ensuring that changes to the interval from Settings take immediate effect. The worker queries HiveDao.getHivesDueForInspection() with the appropriate cutoff timestamp, collects the hive codes of all overdue hives, and posts a single consolidated notification listing them.

11.5 Honey Flow Progress Bar

The Honey Flow Season progress bar on the Harvest screen is implemented using Android's ProgressBar widget with a custom horizontal style. The progress value is computed as (currentSeasonTotalKg / seasonTargetKg) * 100, where seasonTargetKg is a user-configurable value stored in SharedPreferences (default: 10 kg per hive). The bar updates in real time after each harvest entry is saved, providing a visual indicator of progress toward the seasonal honey yield goal.

11.6 Year-over-Year Harvest Comparison

The HarvestDao.getYearlyTotals(hiveId) query groups harvest_entries by the year component of harvestDate and sums quantityKg for each year. The results are displayed in a two-column comparison card showing the current year's total against the previous year's total, with a percentage change indicator. The comparison card is rendered in HarvestActivity using a custom XML layout and populated in the background thread after the DAO query completes.

11.7 Flora Calendar Implementation

The Flora Calendar is implemented as a static data set defined in a Kotlin object (FloraData.kt) containing a list of FloraEntry data classes, each holding monthIndex (1–12), plantName, bloomPeriod, forageType (Nectar/Pollen/Both), and beekeepingTip. FloraCalendarActivity reads the current month from Calendar.getInstance(), highlights the matching month section in the RecyclerView, and scrolls to it on launch. No database or internet access is required for the Flora Calendar.

11.8 Security – PIN Authentication

The 4-digit PIN is stored in SharedPreferences under a named preferences file (BeekeeperPrefs). The login session state (isLoggedIn boolean) is set to true on successful login and cleared to false on logout. LoginActivity checks this flag on creation and routes directly to MainActivity if the session is active. PIN hashing is planned as a security enhancement in a future version.

12. Project File Structure

File / Directory	Purpose
app/build.gradle	App-level Gradle config: plugins, SDK versions, Room, WorkManager, and kapt dependencies.
data/Hive.kt	Room @Entity data class representing a single registered hive.
data/InspectionLog.kt	Room @Entity data class representing a single hive inspection log with alert fields.
data/HarvestEntry.kt	Room @Entity data class representing a single honey harvest record.
data/HiveDao.kt	Room @Dao interface with getAll, getById, insert, update, delete, and getHivesDueForInspection queries.
data/InspectionDao.kt	Room @Dao interface with insert, getByHiveId, and getLastInspectionDate queries.
data/HarvestDao.kt	Room @Dao interface with insert, getByHiveId, and getYearlyTotals queries.
data/AppDatabase.kt	Room @Database singleton with version management and migration definitions.
engine/DecisionMatrix.kt	Stateless Kotlin object implementing the hive health rule evaluation engine.
engine/AlertResult.kt	Data class holding colour (RED/AMBER/GREEN), alert title, and recommendation string.
LoginActivity.kt	PIN creation and authentication screen. Routes to MainActivity on success.
MainActivity.kt	Main dashboard: hive list, summary cards, quick-action buttons, options menu.
HiveFormActivity.kt	Hive registration and edit form: code, location, origin, date, notes.
InspectionActivity.kt	Inspection checklist form. Calls DecisionMatrix on save and displays AlertResult card.
HarvestActivity.kt	Harvest log form with Honey Flow progress bar and year-over-year comparison card.
FloraCalendarActivity.kt	Static flora calendar display scrolled to the current month on launch.

InspectionReminderWorker.kt	WorkManager Worker that checks for overdue inspections and dispatches notifications.
ProfileActivity.kt	Beekeeper profile form: name, apiary name, location, phone. Persists via SharedPreferences.
SettingsActivity.kt	Settings for inspection reminder interval, notification toggle, and change PIN.
res/layout/activity_main.xml	Dashboard layout: header, summary cards, hive RecyclerView, FAB.
res/layout/activity_inspection.xml	Inspection form layout with checklist switches, spinners, and alert card.

File / Directory	Purpose
res/drawable/*.xml	Shape drawables for card backgrounds, alert badges (red/amber/green), avatar circles.
AndroidManifest.xml	App manifest: permissions, activity declarations, worker registration, launcher activity.

13. Permissions & Privacy

Permission	Required For	Notes
POST_NOTIFICATIONS	Inspection overdue alerts	Required on Android 13+. User prompted on first notification trigger.
SCHEDULE_EXACT_ALARM	Precise reminder scheduling	Required on Android 12+. User may need to grant via system settings.
RECEIVE_BOOT_COMPLETED	Restart WorkManager after reboot	Ensures inspection reminders resume automatically after device restart.

No personal user data is transmitted to any external server. All hive records, inspection logs, harvest entries, beekeeper profile information, and PIN data remain exclusively on the beekeeper's device. The application does not collect analytics, crash reports, or usage statistics.

14. Impact Goals

14.1 Sweet Revolution – Increasing Honey Production

By enabling timely, data-driven interventions, Madhu-Marga directly reduces colony losses caused by undetected queen failure, pest infestations, and missed harvest windows. Beekeepers who consistently log inspections and act on Decision Matrix alerts improve their colony survival rates and honey yields, contributing to India's honey production targets under the National Beekeeping and Honey Mission (NBHM).

14.2 Biodiversity – Supporting Pollinator Health

Healthy bee colonies maintained through informed hive management are more effective pollinators of surrounding agricultural crops. By reducing colony loss among small-scale beekeepers, Madhu-Marga indirectly supports the pollination of orchards, vegetable farms, and oilseed crops in the vicinity of apiaries, increasing overall agricultural yield for the farming community.

14.3 Sustainable Income – Rural Livelihood Empowerment

Beekeeping is a low-investment, high-return secondary income activity accessible to marginal farmers, women's self-help groups, and landless agricultural workers. By democratising access to expert-level hive management guidance through an offline Android app, Madhu-Marga lowers the skill barrier for entry into beekeeping and reduces the colony loss risk that often discourages beginners from continuing.

14.4 Digital Inclusion in Agriculture

The application demonstrates that effective agricultural decision-support software does not require a smartphone with high-end specifications, a reliable data connection, or significant digital literacy. By targeting API 21 and above and operating fully offline, the app is accessible to beekeepers using affordable handsets in remote rural Karnataka and across India.

14.5 Scalability of Impact

The modular architecture of Madhu-Marga — with separate data, engine, activity, and worker layers — supports rapid expansion. Future versions can add weather API integration, image-based pest identification, multi-language support, and community knowledge sharing without requiring a fundamental redesign of the existing codebase.

15. Success Criteria

The project will be considered successfully completed when the following criteria are met:

#	Success Criterion	Measurement	Priority
1	App loads and reaches the Dashboard within 800 ms on a reference device (Redmi 9A or equivalent).	Android Profiler measurement	**Critical**
2	A complete hive inspection log can be saved within 5 taps from the login screen.	Manual UI walkthrough	**Critical**

3	Decision Matrix generates a RED alert when Low Activity is logged with no queen or eggs present.	Manual test with Low Activity log	**Critical**
4	All hive, inspection, and harvest records are retained after device reboot and 30+ days of inactivity.	Room DB inspection test	**Critical**
5	PIN login correctly blocks access with wrong PIN and grants access with correct PIN.	Manual security testing	**Critical**
6	Inspection overdue notification is delivered for any hive not inspected within the configured interval.	WorkManager log verification	**High**
7	Honey Flow Season progress bar updates correctly after each harvest entry is saved.	Manual validation	**High**
8	Year-over-year harvest comparison card shows correct totals for current and previous seasons.	Manual data entry and verification	**High**
9	App does not crash during complete walkthrough of all screens and features.	Manual regression testing	**High**
10	All features work fully in Airplane Mode (no internet required for any core function).	Airplane Mode test on device	**Medium**

Conclusion

Madhu-Marga represents a focused and socially meaningful application of Android development in the domain of agricultural hive management. By addressing a clear and well-defined problem — the lack of a reliable, offline, and intelligent digital diary for India's small-scale beekeepers — the project delivers measurable practical value alongside technical merit.

The application's architecture prioritises reliability, simplicity, and offline-first operation above all else. Every design decision, from Room Database for permanent storage to WorkManager for resilient background reminder scheduling and the Decision Matrix Engine for intelligent hive health analysis, is motivated by a single question: will this work for a beekeeper standing beside their hives on a remote farm with a basic smartphone and no internet connection?

The Decision Matrix Engine is the application's defining innovation. By encoding expert beekeeping knowledge into a transparent, rule-based evaluation system, the app delivers the equivalent of an experienced mentor's advice at the moment of inspection — without requiring internet connectivity or cloud AI services. This approach makes the guidance auditable, predictable, and trustworthy for a farming audience that values practicality over complexity.

The project also establishes a strong technical foundation for future expansion. The separation of data, engine, view, and controller layers supports the addition of new features — weather integration, image-

based pest identification, multi-language support, and cloud backup — without requiring a fundamental redesign. The Room migration framework ensures that future schema changes can be deployed to existing users without data loss.

In conclusion, Madhu-Marga is not simply a technical deliverable — it is a small but meaningful step toward sustainable agricultural livelihoods, biodiversity conservation, and digital empowerment for the millions of beekeepers who play a vital and often invisible role in India's food security.